

Isolation Forest

Détection d'Anomalies par Isolation

Cours de Machine Learning Non Supervisé

28 décembre 2025

Table des matières

1 Introduction et Motivation

1.1 Contexte

La détection d'anomalies est cruciale dans de nombreux domaines :

- **Maintenance prédictive** : Détecter les pannes avant qu'elles ne surviennent
- **Cybersécurité** : Identifier les intrusions réseau
- **Finance** : Détecter les fraudes bancaires
- **Santé** : Repérer les pathologies rares

1.2 Limites des Approches Classiques

Les méthodes traditionnelles (K-Means, GMM) modélisent la *normalité* puis considèrent comme anomalies tout ce qui s'en écarte. Problèmes :

1. Nécessitent de définir une distribution normale (hypothèse forte)
2. Coûteuses en calcul pour des données haute dimension
3. Sensibles aux paramètres (nombre de clusters, etc.)

Concept Clé

Paradigme d'Isolation Forest :

Au lieu de modéliser le "normal", on cherche directement à **isoler** les anomalies. L'idée est que les anomalies sont :

- **Rares** (peu nombreuses)
- **Différentes** (valeurs atypiques)

⇒ Elles sont donc **plus faciles à séparer** du reste des données par des partitions aléatoires.

2 Principe de Fonctionnement

2.1 L'Arbre d'Isolation (iTree)

Un **iTree** est un arbre binaire construit de manière aléatoire :

1. **Sélection aléatoire** d'une caractéristique q parmi les d dimensions
2. **Choix aléatoire** d'un seuil de coupure p tel que :

$$\min(X_q) < p < \max(X_q)$$

3. **Partition** des données en deux nœuds enfants selon $X_q < p$ ou $X_q \geq p$
4. **Récursion** jusqu'à ce que :
 - Le nœud ne contienne qu'un seul point (isolé)
 - Ou la hauteur maximale soit atteinte

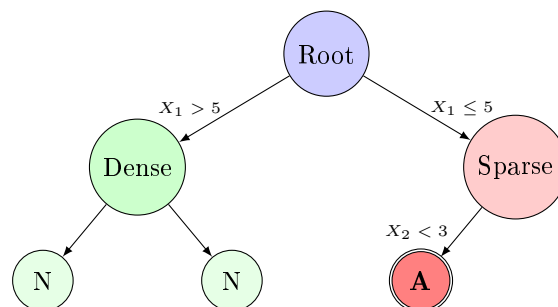


Figure 1 : Exemple d'iTree. L'anomalie (A) est isolée en 2 coupes, les points normaux (N) nécessitent plus de profondeur.

2.2 La Forêt d'Isolation

Une **Isolation Forest** est un ensemble de t iTrees construits sur des sous-échantillons aléatoires des données.

Exemple Pratique

Algorithme de Construction :

1. Fixer le nombre d'arbres t (ex : 100)
2. Pour chaque arbre $i = 1, \dots, t$:
 - Tirer aléatoirement ψ points (ex : 256) du dataset
 - Construire un iTree sur ce sous-échantillon
3. Retourner la forêt $\{T_1, T_2, \dots, T_t\}$

3 Fondements Mathématiques

3.1 Longueur du Chemin

Fondement Mathématique

Définition : Longueur du Chemin

Pour un point x dans un arbre T , la **longueur du chemin** $h(x)$ est le nombre d'arêtes traversées depuis la racine jusqu'à la feuille contenant x .

Propriété Clé :

- Anomalie $\Rightarrow h(x)$ **petit** (isolé rapidement)
- Point normal $\Rightarrow h(x)$ **grand** (nécessite beaucoup de coupes)

3.2 Score d'Anomalie

La longueur moyenne sur tous les arbres doit être normalisée pour être comparable entre datasets de tailles différentes.

Fondement Mathématique

Facteur de Normalisation $c(n)$

Pour un échantillon de taille n , $c(n)$ est la longueur moyenne d'un chemin dans un *Binary Search Tree* (BST) non réussi :

$$c(n) = 2H(n-1) - \frac{2(n-1)}{n}$$

où $H(i) = \ln(i) + \gamma$ est le nombre harmonique ($\gamma \approx 0.5772$ constante d'Euler).

Approximation : $c(n) \approx 2 \ln(n-1)$ pour n grand.

Fondement Mathématique

Score d'Anomalie $s(x, n)$

Le score d'anomalie pour un point x est défini par :

$$s(x, n) = 2^{-\frac{E(h(x))}{c(n)}}$$

où :

- $E(h(x)) = \frac{1}{t} \sum_{i=1}^t h_i(x)$ est la longueur moyenne du chemin sur les t arbres
- $c(n)$ normalise par rapport à la taille de l'échantillon

Interprétation :

$$\begin{aligned} s(x, n) &\rightarrow 1 && \text{Anomalie certaine} \\ s(x, n) &\rightarrow 0 && \text{Point normal} \\ s(x, n) &\approx 0.5 && \text{Indécis (probablement normal)} \end{aligned}$$

3.3 Justification Théorique

Exemple Pratique

Pourquoi $2^{-E(h(x))/c(n)}$?

1. Si $E(h(x)) \approx c(n)$ (longueur typique d'un BST) :

$$s(x, n) = 2^{-1} = 0.5 \quad (\text{comportement aléatoire})$$

2. Si $E(h(x)) \ll c(n)$ (chemin très court) :

$$s(x, n) \rightarrow 1 \quad (\text{anomalie})$$

3. Si $E(h(x)) \gg c(n)$ (chemin très long, impossible en pratique) :

$$s(x, n) \rightarrow 0 \quad (\text{normal})$$

4 Hyperparamètres et Tuning

4.1 Paramètres Principaux (Scikit-Learn)

1. **n_estimators** (défaut : 100)
 - Nombre d'arbres dans la forêt
 - $\uparrow$ Plus stable, mais plus lent
 - Recommandation : 100-200 suffisent généralement
2. **max_samples** (défaut : "auto" = 256)
 - Taille du sous-échantillon pour chaque arbre
 - **Crucial** : Évite le "swamping" (masquage des anomalies par la masse)
 - Recommandation : 256 pour $n > 256$, sinon n
3. **contamination** (défaut : "auto")
 - Proportion attendue d'anomalies
 - Détermine le seuil de décision sur le score
 - Exemple : **contamination=0.05** $\Rightarrow$ 5% des points avec les scores les plus élevés seront classés comme anomalies
4. **max_features** (défaut : 1.0)
 - Fraction de features à considérer pour chaque split
 - Augmente la diversité si < 1.0

Attention

Piège du Swamping :

Si on utilise `max_samples = n` (tout le dataset), les anomalies minoritaires peuvent être "noyées" dans la masse de points normaux, rendant leur isolation difficile.
⇒ Toujours utiliser un sous-échantillonnage (256 est un bon compromis).

5 Méthodes d'Évaluation

5.1 Cas Supervisé (Labels Disponibles)

Lorsque les vraies étiquettes sont connues (comme dans `ai4i2020.csv`) :

1. Matrice de Confusion

	Prédit Normal	Prédit Anomalie
Vrai Normal	TN	FP
Vrai Anomalie	FN	TP

2. Précision (Precision)

$$P = \frac{TP}{TP + FP}$$

Parmi les alertes levées, combien sont vraies ?

3. Rappel (Recall / Sensibilité)

$$R = \frac{TP}{TP + FN}$$

Parmi les vraies anomalies, combien ont été détectées ?

4. F1-Score

$$F_1 = 2 \cdot \frac{P \cdot R}{P + R}$$

Moyenne harmonique (équilibre Précision/Rappel)

5. ROC AUC Aire sous la courbe ROC (Receiver Operating Characteristic)

- Mesure la capacité du modèle à séparer les classes
- Indépendant du seuil choisi

5.2 Cas Non Supervisé (Pas de Labels)

- Analyse de la distribution des scores
- Inspection manuelle des points à score élevé
- Validation métier (expertise domaine)

6 Exemple Pratique Complet

6.1 Dataset : AI4I 2020 Predictive Maintenance

Implémentation Python

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.ensemble import IsolationForest
4 from sklearn.preprocessing import StandardScaler
5 from sklearn.metrics import classification_report, confusion_matrix,
```

```

roc_auc_score
6 import matplotlib.pyplot as plt
7 import seaborn as sns
8
9 # 1. CHARGEMENT DES DONNÉES
10 df = pd.read_csv("ai4i2020.csv")
11
12 # 2. PRÉPARATION
13 # Sélection des features numériques
14 features = ['Air temperature [K]', 'Process temperature [K]',
15             'Rotational speed [rpm]', 'Torque [Nm]', 'Tool wear [min]']
16 X = df[features]
17 y = df['Machine failure'] # 0 = Normal, 1 = Panne
18
19 # Normalisation (important pour IF)
20 scaler = StandardScaler()
21 X_scaled = scaler.fit_transform(X)
22
23 # 3. ENTRAÎNEMENT DU MODÈLE
24 # On estime ~3% de pannes dans le dataset
25 iso_forest = IsolationForest(
26     n_estimators=100,
27     max_samples=256,
28     contamination=0.03, # 3% d'anomalies attendues
29     random_state=42,
30     n_jobs=-1
31 )
32
33 # fit_predict retourne 1 (inlier) ou -1 (outlier)
34 predictions = iso_forest.fit_predict(X_scaled)
35
36 # 4. CONVERSION DES PRÉDICTIONS
37 # IF : 1 = Normal, -1 = Anomalie
38 # Nos labels : 0 = Normal, 1 = Panne
39 y_pred = np.where(predictions == -1, 1, 0)
40
41 # 5. VALUATION
42 print("="*60)
43 print("MATRICE DE CONFUSION")
44 print("="*60)
45 cm = confusion_matrix(y, y_pred)
46 print(cm)
47
48 print("\n" + "="*60)
49 print("RAPPORT DE CLASSIFICATION")
50 print("="*60)
51 print(classification_report(y, y_pred, target_names=['Normal', 'Panne']))
52
53 # 6. CALCUL DU ROC AUC
54 # On utilise les scores (decision_function) pour le AUC
55 scores = iso_forest.decision_function(X_scaled)
56 # Attention : scores négatifs = anomalies, on inverse
57 auc = roc_auc_score(y, -scores)
58 print(f"\nROC AUC Score : {auc:.4f}")
59
60 # 7. VISUALISATION DES SCORES
61 plt.figure(figsize=(12, 5))
62
63 plt.subplot(1, 2, 1)
64 plt.hist(scores[y == 0], bins=50, alpha=0.7, label='Normal', color='blue')
65 plt.hist(scores[y == 1], bins=50, alpha=0.7, label='Panne', color='red')
66 plt.xlabel('Anomaly Score (decision_function)')

```

```

67 plt.ylabel('Fr quence')
68 plt.title('Distribution des Scores IF')
69 plt.legend()
70 plt.grid(True, alpha=0.3)
71
72 plt.subplot(1, 2, 2)
73 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
74 plt.title('Matrice de Confusion')
75 plt.ylabel('Vrai Label')
76 plt.xlabel('Pr diction')
77
78 plt.tight_layout()
79 plt.show()

```

6.2 Résultats Attendus

Exemple Pratique

Interprétation Typique :

- **Recall élevé** (70-80%) : La plupart des pannes sont détectées
- **Précision faible** (10-20%) : Beaucoup de fausses alarmes
- **Trade-off** : En maintenance, on préfère inspecter trop souvent que rater une panne critique

Pour améliorer la précision :

1. Ajuster **contamination** (réduire si trop de FP)
2. Feature engineering (ajouter des features discriminantes)
3. Combiner avec des règles métier

7 Avantages et Limites

7.1 Avantages

- **Efficacité** : Complexité linéaire $O(t \cdot \psi \cdot \log \psi)$ avec $\psi \ll n$
- **Pas d'hypothèse de distribution** : Fonctionne sur données non-gaussiennes
- **Robuste** : Peu sensible aux paramètres
- **Interprétable** : Le score a une signification claire

7.2 Limites

- **Anomalies locales** : Moins performant que LOF pour détecter des anomalies dans des clusters denses
- **Haute dimension** : Performance peut se dégrader si $d \gg 100$
- **Contamination** : Nécessite d'estimer la proportion d'anomalies

8 Conclusion

Isolation Forest est une méthode puissante et efficace pour la détection d'anomalies, particulièrement adaptée aux cas où :

- Les anomalies sont **globalement différentes** (pas seulement localement)
- Le dataset est **volumineux** (scalabilité)
- On ne connaît pas la distribution des données normales

Pour des anomalies plus subtiles (contextuelles, locales), considérer LOF ou des autoencodeurs.